Advanced Curriculum for Agentic Coding and Orchestration on macOS

The software engineering landscape has undergone a fundamental architectural shift, transitioning from localized, syntax-driven development to asynchronous, agentic orchestration. For a technical power user already proficient in conceptual logic, prompt engineering, and the utilization of advanced reasoning models such as Gemini 3.1 Pro via Google AI Studio, the final frontier is environmental mastery. This encompasses achieving absolute fluency in the macOS Unix terminal, the Google Cloud CLI (gcloud), the Gemini CLI, and the Google Antigravity IDE.[1] This curriculum provides an exhaustive, phased architecture for mastering this transition, steering development away from fragile, ad-hoc methodologies toward robust, spec-driven engineering.

Before detailing the phased curriculum, an architectural deconstruction of the legacy "vibe coding" methodology is required. In early 2025, the paradigm of "vibe coding" gained immense popularity, characterized by a highly informal, iterative approach to AI code generation where the developer relinquished control over the codebase's underlying structure.[5] An analysis of the provided visual artifact, "15 Rules of Vibe Coding," reveals that while it served as a foundational stepping stone, many of its heuristics have calcified into severe anti-patterns that induce catastrophic technical debt when applied to enterprise-grade or highly complex applications in 2026.[7]

The transition to fluent agentic coding necessitates the immediate unlearning of these outdated practices and the adoption of modern, deterministic constraints. The table below analyzes the critical flaws in the 2025 heuristics and establishes the 2026 spec-driven alternatives that will serve as the foundation for this curriculum.

2025 "Vibe Coding" Heuristic (Artifact Analysis)	Architectural Flaw & Anti-Pattern Analysis	Modern 2026 Alternative (Spec-Driven Development)
Rule 4: Create new chats in Composer. "Open a new chat for each distinct task. Keep agent chats short."	This heuristic treats agents as stateless, amnesiac entities. Constantly resetting the context window forces the developer to manually re-explain the architectural intent, leading to fragmented, disjointed code logic across the repository.[9]	**The .agent/ Directory & Context Saturation Prevention.** Instead of resetting chats, developers define persistent Rules (.agent/rules/) and declarative Workflows (.agent/workflows/). The agent maintains deep codebase awareness without the noise of conversational history.[10]
Rule 9: Copy errors and paste into Composer agent. "When	Blindly pasting stack traces isolates the error from its	**Standard Stream Piping (stderr).** Error logs are piped

errors occur, copy error messages from your console and paste them..."	environmental execution state. The agent must guess the configuration, leading to hallucination loops where the AI suggests arbitrary package installations or incorrect syntax fixes.[5]	directly from the macOS terminal standard error stream into the Gemini CLI, attaching the exact file state and local environment configuration automatically (e.g., 2>&1 | gemini).[12]
Rule 15: Enjoy the process—just vibe. "Embrace the creative journey of vibe coding, experiment, learn, and have fun... just vibe."	The "just vibe" methodology optimizes for prototype speed at the direct expense of maintainability. It operates on a linear loop of prompt-guess-generate, resulting in unmaintainable legacy output that defies version control and architectural coherence.[8]	**Spec-Driven Development.** The agent is strictly bound to an EARS (Easy Approach to Requirements Syntax) functional specification (spec.md). The AI operates as a compliance engine executing against a deterministic plan, eliminating arbitrary architectural drift.[8]

Mastering the modern stack requires abandoning the unstructured "vibe" and embracing rigorous orchestration. The following three-phase curriculum outlines the precise technical execution required to achieve this fluency.

PHASE 1: The Foundation (Terminal & CLI Literacy)

To achieve absolute fluency in agentic coding loops, the developer must first establish mastery over the environment in which the agents operate. Agents do not function in a vacuum; they interact with the host operating system via shell commands, file descriptors, and standard input/output streams.[13] On macOS, the default shell is Z shell (zsh), a highly customizable Unix shell that acts as the primary interface between the user, the AI agent, and the POSIX-compliant operating system.

Understanding the terminal requires a mental shift from a visual hierarchy (windows and icons) to a text-based, spatial hierarchy. The macOS terminal operates on a file tree starting at the root directory (/). The user's specific domain is the home directory (~ or /Users/username/). When an agent is granted terminal execution rights, it navigates this exact structure. Comprehending the difference between absolute paths (e.g., /Users/username/Projects/app) and relative paths (e.g., ./app or ../) is critical, as context loading in tools like the Gemini CLI depends entirely on specifying the correct directory trees to avoid context saturation.[16]

Visualizing the Invisible OS: 5 Essential Commands

To bridge the conceptual gap between a graphical user interface (GUI) and the command-line interface (CLI), the following five essential commands translate visual actions into terminal execution. Mastery of these commands is required before permitting an AI agent to autonomously manipulate the file system.

Unix Command (zsh)	Graphical GUI Equivalent & Analogy	Advanced Agentic Workflow Execution
ls -la	**The Finder Window.** Equivalent to opening a folder, enabling "Show Hidden Files," and viewing the detailed list (size, date modified).	Agents utilize ls to index directory contents. The -la flag ensures the agent sees critical hidden configuration files (e.g., .env, .agent/rules/) and file permissions, which are invisible in standard graphical views.
cd [path]	**Double-Clicking / Back Button.** Equivalent to double-clicking a folder to enter it, or clicking the "Back" arrow to return to the parent directory.	Critical for scoping an agent's execution context. Running cd../ moves the execution context up one directory. Without strict directory navigation, an agent might overwrite files in the wrong project.
mkdir -p [dir]	**New Folder.** Equivalent to right-clicking and selecting "New Folder."	The -p (parents) flag allows the creation of deeply nested directory trees (e.g., mkdir -p src/components/ui/buttons) in a single command. This is highly utilized by agents when scaffolding new project architectures based on a specification.
mv [src][dest]	**Drag and Drop / Rename.** Equivalent to dragging a file to a new folder, or right-clicking to rename a file.	Used by agents to restructure codebases during refactoring loops. It is imperative to ensure the destination path is perfectly specified, as a hallucinated path will instantly move the file to an unintended location, effectively "losing" it.
cat [file]	**Quick Look.** Equivalent to selecting a file and pressing the Spacebar to instantly view its textual contents without opening a dedicated application.	The foundational command for reading data into an agent's context window. Using the Unix pipe operator (|), a developer can execute cat error.log | gemini to stream the contents of a file directly into the AI's cognitive reasoning loop.[12]

CLI Toolchain: Google CLI and Gemini CLI Setup

Modern agentic workflows on macOS require a dual-CLI configuration. The Google Cloud CLI (gcloud) handles cloud infrastructure, authentication, and deployment states, while the Gemini CLI manages local agentic orchestration, file manipulation, and code generation.[2] These are distinct tools operating at different architectural layers.

Establishing the Google Cloud CLI (gcloud)

The gcloud CLI manages cloud resources and acts as the bridge for deploying the applications generated by local agents. To establish this toolchain on macOS natively, execute the following configurations:

1. Download the Darwin (macOS) binary archive for gcloud (version 557.0.0 or later).[2]
2. Extract the archive within the home directory.
3. Execute the installation script to append the binary to the zsh path: ./google-cloud-sdk/install.sh [18]
4. Initialize the environment and authenticate: gcloud init [2]
5. Set persistent defaults. This is a critical step for agentic workflows. If defaults are not set, deployment commands will pause execution to ask for interactive user input (e.g., "Which region do you want to deploy to?"), which will cause autonomous agent loops to hang indefinitely: gcloud config set project gcloud config set compute/region [19]

Establishing the Gemini CLI The Gemini CLI, functioning as a terminal-native AI assistant, enhances the existing workflow by providing access to the Gemini 3.1 Pro model's 1-million-token context window. It employs a Reason and Act (ReAct) loop that mimics developer problem-solving: analyzing the prompt, executing file system reads, observing the output, and writing code.[13]

To install the Gemini CLI via Homebrew, execute the following command in the macOS terminal: brew install gemini-cli [17]

Once installed, authenticate the CLI using a Google account to access the free tier limits (which provide ample requests for local weekend development) or attach it to a Google Cloud Project for enterprise API usage: gemini login [17]

Pitfall Warning: Destructive Commands and macOS Safety Rails

The macOS terminal operates under a strict trust model: the operating system assumes the user (or the AI agent acting on the user's behalf) possesses explicit intent for every command issued, and it executes them without hesitation or moral judgment.[21] Because AI agents occasionally hallucinate parameters, misunderstand relative file paths, or attempt to force installations, granting them terminal access without establishing safety rails is a catastrophic risk.[21]

The following represent the top three destructive command patterns that beginners and unsupervised agents accidentally execute:

1. **Destructive File Deletion (rm -rf / or rm -rf *)**: The rm command removes files. The -r flag makes it recursive (deleting directories), and -f forces the deletion without asking for

confirmation.[21] If an agent intends to clear a localized build folder but hallucinates the path and executes rm -rf * in the root home directory, the entire user profile is instantly wiped without the possibility of retrieval from the macOS graphical Trash bin.[21]

2. **Permission Disasters (chmod -R 777 /)**: This command recursively grants read, write, and execute permissions to every file for every user on the system.[21] While coding agents frequently suggest this command as a lazy workaround for "Permission Denied" errors during script execution, it completely destroys the macOS Unix security model and can render the operating system permanently unbootable.[21]
3. **Destructive Redirection (> important_file.txt)**: In Unix, the > operator redirects the output of a command into a file. If the file already exists, the > operator instantly truncates (empties) the file before writing the new data.[21] If an agent generates an incomplete script and redirects it into a vital project file, the original working code is destroyed.

Establishing macOS Safety Rails (The ~/.zshrc Configuration)

To protect the host environment from these catastrophic agentic hallucinations, the developer must implement strict guardrails directly into the Z shell configuration file. Open the terminal and execute nano ~/.zshrc, then append the following safety configurations:

Bash

```
# 1. Prevent 'rm -rf *' from silently executing.
# Zsh will pause, output a warning, and ask for explicit human confirmation.
setopt rm_star_silent
setopt rm_star_wait

# 2. Alias the remove command to always prompt for interaction before deleting files.
alias rm="rm -i"

# 3. Prevent overriding files accidentally with output redirection.
# If an agent tries to use '>', zsh will block it if the file exists.
# The agent must explicitly use '>|' to force an overwrite.
setopt noclobber

# 4. Alias chmod to prevent recursive root permission changes, blocking the 777 disaster.
alias chmod="chmod --preserve-root"
```

After saving the file, execute source ~/.zshrc to activate the guardrails.[24] These rails ensure that if the Gemini CLI attempts a destructive action, the terminal will halt execution, preventing the loss of local data.

PHASE 2: The Translation (Vibe Coding Mechanics)

Once the terminal environment is secured, the developer must master the mechanics of translating abstract logic into executable code. As previously established, the 2026 methodology abandons the chaotic "vibe coding" approach in favor of Spec-Driven Development.[8] This methodology utilizes the AI not to guess the architecture through iterative chat, but to strictly implement a predefined functional specification, bridging the gap between high-level logic and low-level terminal execution.

Translating Natural Language Intent into Structured Scaffolding

The process begins by formalizing the "vibe" into an EARS (Easy Approach to Requirements Syntax) compliant Markdown document. This document, typically named spec.md, acts as the single source of truth for all subsequent agentic operations.[8] A robust spec.md for an agentic workflow must explicitly define:

1. **The Objective:** A concise summary of the application's core functionality.
2. **The Technology Stack:** Explicit definition of the language, framework, and package manager (e.g., Python 3.12, Typer CLI framework, standard pip).
3. **Directory Structure:** A visual tree representing the exact file hierarchy the agent must create.
4. **Operational Boundaries:** Explicit instructions on what the agent is *not* allowed to do (e.g., "Do not install global npm packages," "Do not modify the database schema without prompting").

By establishing these deterministic constraints, the developer mitigates the risk of the model's attention budget degrading over time.[26] The agent is forced to continually cross-reference its actions against the spec.md file, ensuring architectural alignment.

Workflow: From Google AI Studio to Terminal Execution

A power-user workflow leverages the distinct strengths of different AI surfaces. The advanced, overarching reasoning of Google AI Studio (utilizing the full capability of the Gemini 3.1 Pro model) is used for conceptual architecture, while the Gemini CLI within the macOS terminal is utilized for localized, headless execution.

Step 1: Architectural Planning (Google AI Studio)

The developer inputs their conceptual logic into the web-based Google AI Studio interface. Because AI Studio is isolated from the local file system, it is the safest place to ideate.

Execution: The developer prompts, "Draft a comprehensive spec.md for a Python-based CLI application that analyzes local CSV files. Define the directory structure, the required libraries (Pandas, Typer), and strict error-handling boundaries."

Step 2: Headless Scaffolding via the Gemini CLI Once the specification is generated in AI Studio, the developer copies the text and saves it locally on their macOS machine as spec.md. The developer then utilizes the Gemini CLI's *headless mode* to orchestrate the scaffolding.[12] Headless mode bypasses the interactive chat interface. It ingests standard input (stdin), executes the AI inference, and prints the result directly to standard output (stdout), allowing the output to be piped into subsequent commands or scripts.[12] The developer opens the macOS terminal and executes the following pipeline:

Bash

```
# Read the spec.md file, pipe it into the Gemini CLI, and instruct the agent to generate a shell script.
# The output is redirected (>) into a new file named scaffold.sh.
cat spec.md | gemini "Read this specification. Output ONLY a valid bash script that creates the directory structure and placeholder files defined in the spec. Do not include markdown formatting or explanations." > scaffold.sh
```

Step 3: Verification and Execution
The developer manually inspects the scaffold.sh file using cat scaffold.sh to ensure no destructive commands were hallucinated. Once verified, the developer grants the script execution permissions and runs it:

Bash

```
chmod +x scaffold.sh
./scaffold.sh
```

This loop ensures that the transition from a natural language prompt to a fully realized project directory is deterministic, reviewable, and instantly executable.

Reading and Debugging Terminal Error Logs via AI Piping

When an agentic build fails or a script throws an exception, the outdated vibe coding heuristic dictates copying the error text from the terminal and pasting it back into a web-based chat interface.[5] This introduces massive friction and strips the error of its environmental context. The fluent 2026 approach utilizes automated debugging pipelines directly within the terminal, leveraging Unix standard streams. When a script fails, the terminal typically outputs standard text to stdout (file descriptor 1) and stack traces or errors to stderr (file descriptor 2).
To debug an issue autonomously, the developer can pipe both streams directly into the Gemini CLI, requesting an automated patch:

Bash

```
# Execute the failing program, redirect standard error (2) to standard output (1)
# Pipe the combined output to the Gemini CLI, instructing it to analyze the failure against the local codebase.
python main.py 2>&1 | gemini "Analyze this stack trace against the current project context. Explain the root cause and output the exact terminal command to patch the dependency."
```

If the agent's logic fails deeper within its own cognitive loop (e.g., the agent repeatedly fails to execute a tool call or ignores a file), the developer must debug the agent itself. Appending the --debug flag to the Gemini CLI invocation exposes the internal ReAct loop.[28] Executing gemini --debug reveals comprehensive verbose logging, including memory usage statistics, detailed context loading information (showing exactly which files were ingested), and step-by-step breakdowns of the tool calls being made to the host OS.[28] Furthermore, utilizing the internal /memory show command allows the developer to inspect the active context window, identifying immediately if the agent is suffering from context saturation due to an excessively large codebase.[28]

PHASE 3: The Orchestration (Agentic Coding in Antigravity IDE)

While the Gemini CLI is unparalleled for terminal-native, single-threaded execution and headless scripting, the development of enterprise-grade, multi-file software requires a dedicated, agent-first environment.[3] Google Antigravity, introduced in late 2025, represents a fundamental evolution of the Integrated Development Environment (IDE). It transitions the IDE from a passive text editor into an active, multi-surface orchestration platform.[1]
Transitioning to Antigravity requires a profound shift in the developer's mindset. The developer must transition their identity from a manual "coder" to an "architect" or "manager," orchestrating a workforce of digital agents that plan, execute, and verify code asynchronously.[31]

Defining the Core Surfaces and the Agentic Loop

Antigravity operates on a three-surface architecture, discarding the singular text-editor view of legacy IDEs [29]:

1. **The Editor View:** A highly advanced, AI-powered synchronous coding environment for manual intervention, featuring inline command generation and tab-completions.[29]
2. **The Agent Manager:** The "Mission Control" surface. This is a dedicated orchestration interface where the developer spawns, monitors, and interacts with multiple agents operating in parallel across different workspaces.[29]
3. **The Browser Subagent:** An embedded, autonomous browser capable of executing end-to-end testing, reading dashboard documentation, and validating UI changes visually without human interaction.[10]

The Autonomous Loop When a developer initiates a complex task within the Agent Manager (e.g., "Implement a secure authentication middleware based on spec.md"), the Antigravity agent engages in a distinct, multi-artifact loop [10]:

1. **Planning:** Before writing a single line of code, the agent generates an *Implementation Plan* and a structured *Task List*. The developer reviews this plan asynchronously, ensuring it aligns with the architectural intent.[10]
2. **Execution and Terminal Actuation:** The agent writes the code and executes terminal

commands (e.g., installing dependencies, running build scripts). Terminal execution is governed by three strict modes:

- *Off:* The agent must ask for manual human approval for every single terminal command.[31]
- *Auto:* The agent autonomously decides whether a command is safe to run or if it requires human permission.[31]
- *Turbo:* The agent automatically executes all terminal commands unless explicitly blocked by a user-defined Deny List.[31] *For transitioning power users, "Auto" mode is strictly recommended, relying on the underlying ~/.zshrc safety rails to catch catastrophic anomalies.*

3. **Validation:** The agent invokes the Browser Subagent to open the local development server, interact with the application, and capture *Screenshots* demonstrating the before-and-after state.[10]
4. **Reporting:** The agent compiles a *Walkthrough* artifact—a summary of the code diffs, the terminal outputs, and the visual screenshots—for the developer's final review.[10]

Best Practices for Version Control and State Management

When autonomous agents possess the authority to modify massive codebases, traditional monolithic file structures fail catastrophically. Agents struggle with "cross-cutting concerns," where related logic is scattered across dozens of nested files (e.g., separating controllers, models, and views into distant root directories). This fragmentation leads to context window exhaustion and degraded reasoning, known colloquially as the "40% Rule".[32]

To manage agentic state, codebases must be structured using **Feature-Driven Vertical Slices**.[32] All logic pertaining to a specific feature must be collocated in a single directory. This enables the agent to load a highly cohesive, narrow slice of the codebase into its context window, vastly improving recall and eliminating merge conflicts.[32]

The .agent Directory: Establishing Deterministic Guardrails State management and behavioral constraints are explicitly defined and version-controlled within the .agent/ directory at the root of the project repository. This directory acts as the central nervous system for Antigravity agents, consisting of Rules, Workflows, and Skills.[10]

Agentic Component	Directory Path	Architectural Function & Execution Paradigm
Rules	.agent/rules/	Passive, persistent constraints injected into the system prompt. They restrict *how* the agent operates globally. For example, a rule file named security.md might state: "Never commit API keys. Always use strict typing." They are

		executed via *Always-On*, *Glob Pattern* matching (e.g., only applying to .ts files), or *Model Decision*.[11]
Workflows	.agent/workflows/	Active, sequential scripts that guide the agent through complex, multi-step actions. Saved as markdown files, they allow the developer to trigger rigorous, standardized loops (e.g., generating documentation post-deployment) by invoking /workflow-name in the chat.[10]
Skills	Managed via IDE	Ephemeral, lightweight scripts that teach the agent *how* to interact with external tools without the heavy operational overhead of persistent Model Context Protocol (MCP) servers. A Database Migration Skill tells the agent exactly how to format a connection string before executing a query.[11]

To maintain rigorous version control when collaborating with agents, a core Workspace Rule must be established. The developer should create a file at .agent/rules/git-protocol.md containing the following directive:

Git Commit Protocol

Activation: Always On
Description: Ensure granular, atomic commits that prevent repository corruption.
Directives:

1. Do not bundle multiple, unrelated features into a single massive commit.
2. Prior to executing git commit, invoke the git-commit-formatter Skill to analyze the git diff and generate a Conventional Commits compliant message.
3. Pause for explicit user approval before executing git push origin main.

The "Fluent" Milestone: A Capstone Exercise

To definitively prove fluency across the macOS terminal, Gemini CLI pipelines, and Antigravity IDE orchestration, the user must complete a comprehensive capstone project. This exercise avoids trivial, static web design, focusing instead on system architecture, autonomous data

piping, and multi-agent coordination.

The Capstone: Autonomous Market Intelligence Aggregator

The objective is to architect a Python-based application that autonomously fetches industry news from RSS feeds, utilizes a local SQLite database for structured storage, and pipes the raw data through the Gemini CLI for sentiment analysis and summarization.

Step 1: Specification and Rules Generation

Using Google AI Studio, generate a rigorous spec.md outlining the Python architecture and SQLite schema. Open the Antigravity IDE, create a new workspace, and establish the .agent/rules/ directory. Create a deterministic rule (.agent/rules/db-schema.md) forcing the Antigravity agent to *always* use parameterized queries during code generation to prevent SQL injection vulnerabilities.

Step 2: Scaffolding via Workflows

Within Antigravity, define a workflow in .agent/workflows/init-project.md containing a rigorous sequence of prompts:

1. "Read spec.md and scaffold the Python environment using venv."
2. "Create the SQLite initialization scripts."
3. "Generate Pytest unit tests for the database connection layer." Execute this initialization sequence by typing /init-project in the Agent Manager interface.[33]

Step 3: Headless CLI Integration Instruct the Antigravity agent to integrate the Gemini CLI directly into the application's Python execution flow using the subprocess module. The Python application will fetch an RSS feed, save the raw text, and pipe it to the Gemini CLI in headless mode to generate a structured JSON summary.[12]

The Antigravity agent must generate the following underlying terminal execution logic within the Python script:

Bash

```
# Capstone snippet executed by the Python application
cat raw_feed.txt | gemini "Summarize this text into strictly formatted JSON with keys: title, sentiment, key_points. Do not output markdown code blocks." --json > summary.json
```

Step 4: Subagent Verification and Final Review Instruct the Antigravity Browser Subagent to autonomously execute the compiled Python application within the integrated terminal. The Subagent will observe the SQLite database output using a local database viewer extension, verifying that the JSON summaries generated by the Gemini CLI pipeline are correctly populated into the database without syntax errors or data truncation.[36]

Completing this capstone demonstrates absolute mastery over the modern 2026 AI workflow. It requires the developer to define strict Z shell guardrails to prevent terminal destruction, utilize headless piping for seamless text translation across standard streams, and orchestrate asynchronous agents to architect, build, and verify a robust, multi-layered application in a highly controlled, spec-driven environment.

Works cited

1. Introducing Google Antigravity, a New Era in AI-Assisted Software Development, accessed on February 22, 2026, https://antigravity.google/blog/introducing-google-antigravity
2. gcloud CLI overview | Google Cloud SDK, accessed on February 22, 2026, https://docs.cloud.google.com/sdk/gcloud
3. Google Antigravity vs Gemini CLI: Agent-First Development vs Terminal-Based AI (2026), accessed on February 22, 2026, https://www.augmentcode.com/tools/google-antigravity-vs-gemini-cli
4. Gemini 3.1 Pro: A smarter model for your most complex tasks, accessed on February 22, 2026, https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/
5. what-is-vibe-coding - HackMD, accessed on February 22, 2026, https://hackmd.io/rFpO7ArHSGC5l81mZi3rXg
6. AI Coding Tools Comparison 2026: The Complete Guide to Vibe Coding, accessed on February 22, 2026, https://www.vibecodingacademy.ai/blog/ai-coding-tools-comparison-2026
7. The 12 Best Vibe Coding Tools in 2026: Build Apps Without Writing Code - NinjaTech AI, accessed on February 22, 2026, https://www.ninjatech.ai/blog/the-12-best-vibe-coding-tools-in-2026
8. Beyond vibe coding: the case for spec-driven AI development - The New Stack, accessed on February 22, 2026, https://thenewstack.io/vibe-coding-spec-driven/
9. From “Vibe Coding” to Spec-Driven Development | by Hamman Samuel, PhD | Medium, accessed on February 22, 2026, https://hammansamuel.medium.com/from-vibe-coding-to-spec-driven-development-c9aec008a81c
10. Getting Started with Google Antigravity, accessed on February 22, 2026, https://codelabs.developers.google.com/getting-started-google-antigravity
11. Tutorial : Getting Started with Google Antigravity Skills - Medium, accessed on February 22, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-antigravity-skills-864041811e0d
12. Automate tasks with headless mode - Gemini CLI, accessed on February 22, 2026, https://geminicli.com/docs/cli/tutorials/automation/
13. Mastering the Gemini CLI. The Complete Guide to AI-Powered... | by Kristopher Dunham | Medium, accessed on February 22, 2026, https://medium.com/@creativeaininja/mastering-the-gemini-cli-cb6f1cb7d6eb
14. Vibe Coding vs Spec-Driven Development: Intent to Implementation Deviation and Context Engineering, accessed on February 22, 2026, https://intent-driven.dev/blog/2025/12/15/vibe-coding-vs-spec-driven-development/
15. Building tech-verdict: A Multi-Agent Decision Engine with KIRO's Spec-Driven Methodology, accessed on February 22, 2026,

https://builder.aws.com/content/387dEQGuyzMwtpbrQ6kD5wVFkZ4/building-tech-verdict-a-multi-agent-decision-engine-with-kiros-spec-driven-methodology
16. Google Gemini CLI: Complete Installation and Usage Guide | WISeAgent Documentation, accessed on February 22, 2026, https://wiseagent.github.io/blogs/docs/GenAI/gemini/cli-guide/
17. google-gemini/gemini-cli: An open-source AI agent that brings the power of Gemini directly into your terminal. - GitHub, accessed on February 22, 2026, https://github.com/google-gemini/gemini-cli
18. Quickstart: Install the Google Cloud CLI, accessed on February 22, 2026, https://docs.cloud.google.com/sdk/docs/install-sdk
19. Getting Started with Google Cloud SDK (gcloud CLI) | by Precious Kweku Obeng - Medium, accessed on February 22, 2026, https://medium.com/@kayobgh/getting-started-with-google-cloud-sdk-gcloud-cli-2ff92a79c148
20. How to Install Google Gemini CLI in MacOS Terminal - YouTube, accessed on February 22, 2026, https://www.youtube.com/watch?v=IJG1Vs0glzI
21. Dangerous Linux Commands Beginners Must Avoid (Top 10) - OSTechNix, accessed on February 22, 2026, https://ostechnix.com/dangerous-linux-commands-beginners/
22. Guardrails for Agentic Coding: How to Move Up the Ladder Without Lowering your Bar, accessed on February 22, 2026, https://jvaneyck.wordpress.com/2026/02/22/guardrails-for-agentic-coding-how-to-move-up-the-ladder-without-lowering-your-bar/
23. Dangerous Linux Commands You Should Never Use in Production | by Harold Finch, accessed on February 22, 2026, https://medium.com/@haroldfinch01/dangerous-linux-commands-you-should-never-use-in-production-098312c947dd
24. Using Z Shell on Macs with the Learn Enough Tutorials, accessed on February 22, 2026, https://news.learnenough.com/macos-bash-zshell
25. using 'rm -rf' safely [duplicate] - Super User, accessed on February 22, 2026, https://superuser.com/questions/1477579/using-rm-rf-safely
26. How to write a good spec for AI agents - Addy Osmani, accessed on February 22, 2026, https://addyosmani.com/blog/good-spec/
27. How to write a great agents.md: Lessons from over 2,500 repositories - The GitHub Blog, accessed on February 22, 2026, https://github.blog/ai-and-ml/github-copilot/how-to-write-a-great-agents-md-lessons-from-over-2500-repositories/
28. How do I get verbose logs from Gemini CLI? - Milvus, accessed on February 22, 2026, https://milvus.io/ai-quick-reference/how-do-i-get-verbose-logs-from-gemini-cli
29. Build with Google Antigravity, our new agentic development platform, accessed on February 22, 2026, https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/
30. Google Antigravity Documentation, accessed on February 22, 2026,

https://antigravity.google/docs/home
31. Tutorial : Getting Started with Google Antigravity | by Romin Irani - Medium, accessed on February 22, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-google-antigravity-b5cc74c103c2
32. Coding Agents as a First-Class Consideration in Project Structures - DEV Community, accessed on February 22, 2026, https://dev.to/somedood/coding-agents-as-a-first-class-consideration-in-project-structures-2a6b
33. Rules / Workflows - Google Antigravity Documentation, accessed on February 22, 2026, https://antigravity.google/docs/rules-workflows
34. Gemini CLI Weekly Update [v0.26.0]: Skills, Hooks and the ability to take a step back with /rewind #17812 - GitHub, accessed on February 22, 2026, https://github.com/google-gemini/gemini-cli/discussions/17812
35. I built a free terminal tool that answers any question with live web sources — free to run, MIT licensed : r/SideProject - Reddit, accessed on February 22, 2026, https://www.reddit.com/r/SideProject/comments/1rbbjk8/i_built_a_free_terminal_tool_that_answers_any/
36. Google Antigravity Tutorial for Beginners - YouTube, accessed on February 22, 2026, https://www.youtube.com/watch?v=1Q5sWISByuw